

SNS化ロードマップ（TODO）

えらぶノートを「自分の記録帳」から「好きなものをみんなと共有できるアプリ」に広げるための TODO。
Xのように、自分のページでは自分の持ち物（評価・レビュー）だけを見られ、タイムラインではみんなの
口コミを見られる形を目指す。

決定事項（2026-10-06 時点）

論点	決定内容
商品の持ち方	共通の 商品（products） と、個人の 持ち物（items） に分ける。商品は検索して選び、なければ自分で登録する
画像	商品は代表画像を持つ。持ち物は自分の画像を任意で持つ。表示は「自分の画像 → なければ商品の代表画像」の順
商品情報の編集権限	最初に登録した人だけが編集できる
カテゴリ	商品には運営が用意した 共通カテゴリ をつける。今の個人カテゴリは自分の持ち物の整理用に残す
公開範囲	持ち物ごとに公開スイッチを持つ。 初期値は非公開
MVP のソーシャル機能	いいね / フォロー / 気になる（保存）
LINE 通知	定期まとめ通知（例: 毎週、なくなりそうな持ち物の一覧を送る）

未決定事項（後で決める）

- 公開時に他の人へ見せる項目
 - 候補: 自分の画像、購入価格（容量あたりのコスト）、メモ、よく使う/手放した の状態
 - **決まるまでは「商品情報・評価・レビュー」だけを公開する**（安全側に倒す）
- 定期まとめ通知の曜日・時刻、通知のオン/オフ設定の粒度
- ゲストユーザーの扱い（公開・いいね・フォローを許可するか）。仮の方針: **ゲストは閲覧のみで、公開・いいね・フォローは不可**
- 共通カテゴリの一覧（例: スキンケア / メイク / ヘアケア / ボディケア / 日用品 / その他）

Phase 1: 商品（products）と持ち物（items）の分離

一番大きな作り変え。ここが固まらないと後の機能が作れないので最初にやる。

DB

- `common_categories` テーブル（共通カテゴリ）を作成し、seed で初期データを投入
- `products` テーブルを作成
 - `name`, `brand_name`, `capacity`, `capacity_unit`, `common_category_id`, `created_by_id` (`users` への FK)
 - 代表画像は Active Storage (`has_one_attached :image`)
 - 検索用に `name` / `brand_name` ヘインデックス
- `items` に `product_id` を追加（移行が終わるまでは null 許可）

データ移行（既存データを壊さないよう段階的に）

- 既存の `items` から `products` を作るデータ移行タスクを作る（rake タスク推奨。再実行しても重複しないようにする）
 - 商品名 + ブランド名が同じものは同じ product にまとめる
 - 作成者 (`created_by_id`) は、その `items` の持ち主にする
 - 既存の画像は **持ち物側の画像のまま残す**（勝手に共通の代表画像にしない）
- 本番相当のデータで移行を試し、件数が合うことを確認する
- 移行後に `items.product_id` を NOT NULL にする
- `items` から `name` / `brand_name` / `capacity` / `capacity_unit` を削除する（**画面がすべて product 参照に切り替わってから**、別のマイグレーションで行う）

Model

- `Product`: バリデーション、`belongs_to :created_by`、`has_many :items`、`editable_by?(user)`
- `Item`: `belongs_to :product`、表示用画像を返すメソッド（自分の画像 → 商品画像の順）
- 容量あたりのコストの計算元を `product.capacity` に変更
- ゲストのサンプルデータ (`User#seed_guest_sample_data!`) を `products` 対応にする

Controller / View

- 持ち物登録フォームを「商品を検索して選ぶ → なければ新規登録」の流れに変更
- オートコンプリートを `products` 検索に変更（自分の持ち物ではなく、全商品が対象）
- 商品情報の編集画面（登録者だけが開けるようにする）
- 一覧・詳細・ホーム・レビュー画面の表示を product 参照に変更

テスト

- `Product` モデルのテスト（バリデーション、編集権限）
- データ移行タスクのテスト（同じ商品がまとまること、再実行しても増えないこと）
- 他人が商品を編集できないこと（controller テスト）

Phase 2: 公開設定とプロフィールページ

DB

- `items` に `public` (boolean, default: false, null: false) を追加
- `users` に `username` (一意、URL 用。例: `/users/mai`)、`bio` (自己紹介) を追加
 - 今の `name` は重複可なので、URL には使わない
 - 既存ユーザーには仮の `username` を自動で割り当てる

Model

- `Item.publicly_visible` スコープ (公開 かつ 投稿者がゲストでない)
- 公開項目を決めるまでは、公開ページで出す項目をメソッドなどで一箇所にまとめておく (後で項目を増やしやすくする)

Controller / View

- 持ち物の詳細・編集画面に公開スイッチを追加
- プロフィールページ `/users/:username` (その人の公開中の持ち物だけを表示)
- 公開持ち物の詳細ページ (ログインしていなくても見られるようにするかを決める)
- プロフィール編集 (`username / bio`)
- **権限まわりの確認:** 今は全画面が `current_user.items` 前提なので、他人のデータを見せる画面は別コントローラ (例: `Public::ItemsController`) に分け、非公開データが漏れないようにする

その他

- 利用規約・プライバシーポリシーを、投稿の公開を前提とした内容に更新

テスト

- 非公開の持ち物が他人のプロフィール・詳細ページに出ないこと
- ゲストの持ち物が公開されないこと

Phase 3: タイムラインと商品ページ

- タイムライン「みんな」: 公開中の持ち物 (評価済みのもの) を新しい順に表示
- タイムラインの絞り込み: 共通カテゴリ、星評価
- 商品ページ `/products/:id`: その商品についての公開口コミの一覧と平均評価
- 商品ページから「自分も持っている → 持ち物に追加」できる導線

- ナビゲーションにタイムラインを追加（ホーム・持ち物・レビューとの位置づけを整理）
 - N+1 対策（`includes(:product, :user)`）とページネーション
-

Phase 4: いいね・フォロー・気になる

いいね

- `likes` テーブル（`user_id`, `item_id`、組み合わせで一意）
- いいねボタン（Turbo Stream でページ遷移なしに切り替え）
- 自分の持ち物・非公開の持ち物にはいいねできないこと

フォロー

- `follows` テーブル（`follower_id`, `followed_id`、組み合わせで一意、自分自身は不可）
- プロフィールページにフォローボタン、フォロー数・フォロワー数
- タイムラインに「フォロー中」タブを追加

気になる（保存）

- `bookmarks` テーブル（`user_id`, `product_id`）
 - 口コミ単位ではなく **商品単位** で保存すると「次を買うものリスト」として使いやすい
- 「気になる」一覧ページ
- 気になる → 実際を買ったら持ち物に追加、の導線

テスト

- 各テーブルの一意制約、権限（非公開・ゲスト・自分自身）
-

Phase 5: LINE で共有

- 公開中の持ち物の詳細ページに「LINE で送る」ボタン（LINE の共有用 URL を使い、API 連携は不要）
 - 持ち物ごとの OGP（タイトル・画像・説明）を設定し、LINE に貼ったときにカードとして表示されるようにする
 - 非公開の持ち物では共有ボタンを出さない
 - （任意）X への共有ボタン
-

Phase 6: LINE 定期まとめ通知

今の LINE 連携は「LINE ログイン」だけで、メッセージ送信には別途 **Messaging API**（LINE 公式アカウント）の準備が必要。

事前準備・調査

- LINE Developers で Messaging API チャンネルを作成（今の LINE ログインチャンネルと同じプロバイダーにする。違うプロバイダーだとユーザー ID が一致しない）
- ユーザーに公式アカウントを友だち追加してもらう導線（友だちでないと送れない）
- 無料プランの月間送信数の上限を確認し、まとめ通知の頻度を決める
- 定期実行の方法を決める（候補: Render Cron Job / GitHub Actions の schedule / Solid Queue の定期ジョブ）。ゲストアカウントの削除（`guest:cleanup`）の定期実行とまとめて決める

実装

- `users` に通知設定（例: `line_notify_enabled`）を追加し、設定画面でオン/オフできるようにする
 - 送信処理をサービスクラスまたはジョブにまとめる（例: なくなりそうな持ち物が1件以上あるユーザーに一覧を送る）
 - 定期実行の設定
 - 送信失敗時のログ出力（ブロックされた、など）
 - テスト（LINE API はスタブにする）
-

後回し（MVP 後に検討）

- バーコード（JAN コード）で商品を登録・検索（`products` に `jan_code` を一意キーとして持たせ、重複登録を防ぐ）
- 重複した商品の統合（管理者機能）
- 通報・ブロック機能（公開ユーザーが増えたら必須）
- コメント機能
- 通知（いいね・フォローされたとき）
- 購入リマインド通知（README の「今後の実装予定」より）